AegonKV: A High Bandwidth, Low Tail Latency, and Low Storage Cost KV-Separated LSM Store with SmartSSD-based GC Offloading

Zhuohui Duan, Hao Feng, Haikun Liu, Xiaofei Liao, Hai Jin, and Bangyu Li,
Huazhong University of Science and Technology

https://www.usenix.org/conference/fast25/presentation/duan

This paper is included in the Proceedings of the 23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings of the 23rd USENIX Conference on File and Storage Technologies is sponsored by

NetApp®

AegonKV: A High Bandwidth, Low Tail Latency, and Low Storage Cost KV-Separated LSM Store with SmartSSD-based GC Offloading

Zhuohui Duan, Hao Feng, Haikun Liu*, Xiaofei Liao, Hai Jin, Bangyu Li

National Engineering Research Center for Big Data Technology and System,
Service Computing Technology and System Lab/Cluster and Grid Computing Lab,
School of Computer Science and Technology, Huazhong University of Science and Technology, China

Abstract

The key-value separation is renowned for its significant mitigation of the write amplification inherent in traditional LSM trees. However, KV separation potentially increases performance overhead in the management of Value region, especially for *garbage collection* (GC) operation that is used to reduce the redundant space occupation. In response, many efforts have been made to optimize the GC mechanism for KV separation. However, our analysis indicates that such solution based on trade-offs between CPU and I/O overheads cannot simultaneously satisfy the three requirements of KV separated systems in terms of throughput, tail latency, and space usage. This limitation hinders their real-world application.

In this paper, we introduce AegonKV, a "three-birds-one-stone" solution that comprehensively enhances the throughput, tail latency, and space usage of KV separated systems. AegonKV first proposes a SmartSSD-based GC offloading mechanism to enable asynchronous GC operations without competing with LSM read/write for bandwidth or CPU. AegonKV leverages offload-friendly data structures and hardware/software execution logic to address the challenges of GC offloading. Experiments demonstrate that AegonKV achieves the largest throughput improvement of 1.28-3.3 times, a significant reduction of 37%-66% in tail latency, and 15%-85% in space overhead compared to existing KV separated systems.

1 Introduction

Key-Value storage, based on the *Log-Structured Merge Tree* (LSM-tree), assumes a pivotal role in contemporary data-intensive applications, primarily owing to its commendable write performance [1, 2]. Nevertheless, the substantial read and write overhead stemming from compaction has persistently posed a formidable challenge within the industry. An extensively embraced optimization involves the adoption of a KV separation architecture [3–9]. This architecture decouples the management of values from the LSM structure, relocating them to a distinct storage area, while concurrently preserving an ordered key sequence within the LSM structure. The implementation of the KV separation architecture markedly mitigates the compaction overhead, as compaction is no longer burdened with the need to process irrelevant value data, and concurrently, the volume of the LSM tree experiences a substantial reduction. These lead to significant performance improvements, particularly in scenarios necessitating extensive storage of large values [10, 11].

To mitigate elevated space redundancy and ensure optimal data retrieval performance, the KV separation architecture incorporates GC mechanism in the management of the Value region [3]. It consolidates files containing a substantial proportion of expired data into newly ordered files, arranged by key and encompassing the latest version data. This operation is directed toward the dual objectives of reclaiming storage space and augmenting scan performance [8, 12]. However, the integration of GC mechanism introduces additional I/O and computational overhead [4]. It is worth noting that existing work in this area focuses on architectural optimization of KV data organization, focusing only on the trade-off between computational and I/O overheads of GC, making it difficult to simultaneously excel in all three aspects of throughput, tail latency, and storage overhead [13–16]. Upon analyzing these works, we observe that the absence of structured scheduling and computational optimization in GC design can intermittently hamper system throughput, but sacrificing GC timeliness or accuracy for system performance can lead to exponential increases in redundant space usage, or significant increases in compaction overhead and write stalls. Therefore, optimizing GC operations to achieve excellent performance in all aspects of KV separation remains a central challenge.

We find that computation and I/O co-optimization of GC operations is possible on SmartSSD, which provides an opportunity to build KV separated systems with high bandwidth, low tail latency, and low storage overhead. Considering that

*Corresponding author: Haikun Liu (hkliu@hust.edu.cn). This work is supported jointly by National Key Research and Development Program of China under grant No.2022YFB4500303, National Natural Science Foundation of China (NSFC) under grants No.62302178, 62332011, and China Postdoctoral Science Foundation under grants No.2023M731195, BX20230134.

SmartSSD integrates an FPGA within the SSD and establishes a pathway to interact with the SSD via an internal bus, this integration of independent internal computational capabilities opens up new possibilities to address challenges related to I/O and computational overhead [17–21]. When GC operations are offloaded to SmartSSD, the internal independent bandwidth can be utilized to reduce host bandwidth contention, and the pipeline of GC execution can be designed by FPGA to alleviate the host computational pressure [22], thus improving the GC performance bottleneck. In summary, we propose for the first time to utilize SmartSSD-based GC offloading in order to simultaneously optimize the throughput, tail latency, and storage overhead of KV separated systems.

However, several challenges arise in utilizing SmartSSD for the offloading of the GC process. In contrast to compaction, GC demands additional information from external sources to aid in determining data validity [5]. Conventional approaches entail the inability to entirely migrate and implement the LSM query within the offloading environment. While the internal bandwidth of SmartSSD can adeptly accommodate a substantial portion of the I/O requests associated with GC, there persists a requirement to transmit specific information back to the LSM. The scheduling and management of this transmission demand careful reconsideration. Furthermore, the FPGA embedded within SmartSSD is inherently limited [23], and an excessive deployment of offloading units encounters resource contention issues, potentially culminating in system crashes. The nuanced equilibrium between computational offloading and the prevailing resource constraints on the SmartSSD FPGA necessitates meticulous consideration to ensure the stability and optimal efficiency of the system.

To surmount these challenges, we introduce AegonKV [24], a "three-birds-one-stone" solution that comprehensively enhances the throughput, tail latency, and space usage of KV separated systems. For the first time, AegonKV proposes to fully utilize SmartSSD for GC offloading without competing with LSM read/write for bandwidth or CPU resources, thus providing a consistently stable and reliable high-throughput service. The fundamental design philosophy underpinning AegonKV revolves around the strategic synergy between software and hardware components. On the FPGA hardware front, we conceptualize and implement a meticulously crafted pipelined GC process. This process encompasses P2P read and write operations, file decoding/encoding, expired data filtering, and merging. Concurrently, we devise a mechanism for offloading GC tasks, incorporating flexible and non-blocking strategies for task data preparation and feedback. In the overarching LSM architecture, we synthesize and integrate insights gleaned from prior GC optimization endeavors. Building upon the foundational practice of calculating invalidation rates through compaction results, we undertake a granular exploration into specific positions. This entails the establishment of a memory-friendly and low-overhead data structure, meticulously designed to track the real-time status of invalidation. This nuanced approach facilitates a more streamlined selection of target files and lends support to an adaptive GC process seamlessly integrated with the hardware.

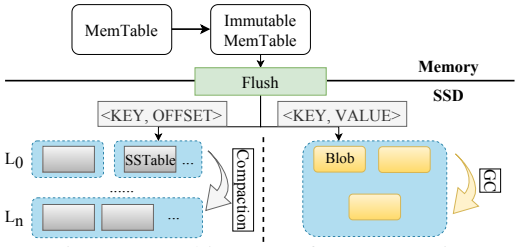

Figure 1: Architecture of KV separation

AegonKV is built on the representative KV separated system-Titan [5]. To the best of our knowledge, this is the first attempt to optimize KV separation by leveraging a near-data-processing architecture. Subsequently, we subject AegonKV to a comprehensive evaluation encompassing both YCSB and realistic production workloads, comparing its performance against Titan, DiffKV [8], BlobDB [7], and RocksDB [25]. The findings underscore that AegonKV achieves the largest throughput improvement of 1.28-3.3 times, a significant reduction of 37%-66% in tail latency, and 15%-85% in space overhead compared to existing KV separated systems.

2 Background

2.1 KV-Separated LSM-tree KV Store

KV Separation Structure. Keys and values are stored separately in KV separated systems [3] as depicted in Figure 1. The Key region still adheres to a structure identical to traditional LSM storage, encompassing MemTables in memory and on-disk storage recognized as SSTables [26]. However, SSTables no longer function as data storage; instead, they store key and references to the address of value. Upon MemTable reaching its capacity, the flush thread segregates the values and appends them to the Value region (referred to as vLog). Concurrently, each value's offset in the file is inscribed to the SSTable along with its corresponding key. The magnitude of the LSM region is significantly diminished by several orders when compared to traditional architectures. This ingenious reduction in space mitigates the read and write amplification issues associated with compaction [4]. Despite the division of read and write processes into two stages, KV separation systems leverage optimizations tailored for random reads and sequential writes on modern SSDs [3, 8]. Consequently, the overall read and write throughput tends to surpass that of traditional LSM architectures.

Garbage Collection. However, the Value region, organized in a log-structured manner, confronts the challenge of accumulating expired data excessively. There are various ways of organizing the Value region, such as the hash-based approach [4] , key-range-based technique [12] and the widely adopted method in Titan [5] and PinK [9] which just splits

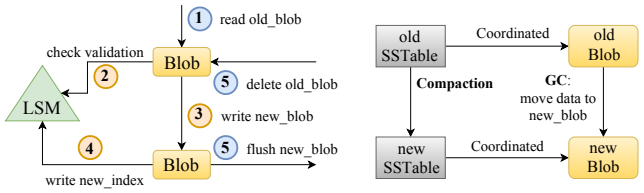

Figure 2: GC process of KV separation

vLog into smaller ones called *Blob file*. Such partitioning facilitates the wise selection of areas with high-benefit potential for GC, consequently reducing additional I/O overhead. The GC process is depicted in Figure 2(a), which we term as *direct GC*. During GC, the entire Blob file is scanned, and at each offset position in the file, the GC thread queries the LSM to determine the validity of the corresponding value. Invalid values are filtered out, while valid ones are subsequently rewritten to a new Blob file, and the new addresses are rewritten back to the LSM (termed *GC metadata*).

While the GC process shares similarities with compaction, their objectives remain distinct. Compaction is mainly used to maintain optimal read performance by shaping the LSM tree, and the impact on throughput is due to write stalls. In contrast, the GC process is theoretically not related to foreground read and write performance. This is because the purpose of GC is to shape the Value region, and read and write of value are done through the offset address with no conflict. However, the current GC still needs assistant data in the LSM region, which actually causes an impact on system performance. Another body of work, represented by studies such as DiffKV [8] and BlobDB [7], seeks to transform the single-layer of Blob files into a multi-layer LSM structure akin to the Key region and maintains a many-to-many relational mapping from SSTables to Blobs. That is, as shown in the Figure 2(b), all the data corresponding to the keys in one SSTable are stored in one or several Blob files. When compaction is performed on the SSTable, accordingly, the value in the original Blob file corresponding the key that is still valid after the compaction merge will be migrated to a new Blob and written to the new SSTable with the new address index. This method is known as *Compaction-Triggered-GC*. Although it increases the overhead of the compaction, it reduces the bandwidth usage of the GC without the need to check and write back to the LSM, thus improving overall performance.

2.2 SmartSSD Computational Storage Device

Recently, near-data processing architectures have been deployed in scenarios grappling with data computation bottlenecks [27]. Samsung's SmartSSD device stands out as the industry's inaugural customizable and programmable computational storage [28]. The SmartSSD architecture seamlessly integrates FPGA and SSD via a rapid direct data movement channel, facilitating efficient near-data processing. By exposing NVMe, FPGA, and FPGA DRAM to distinct host PCIe

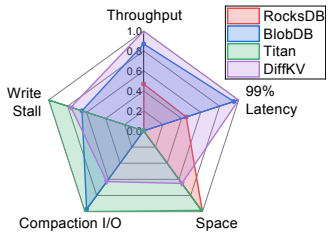

Figure 3: The throughput, 99% latency, space, compaction I/O, write stall capability comparison of RocksDB, BlobDB, Titan, and DiffKV (A larger value implies stronger capability)

address spaces, the device driver judiciously directs diverse requests to respective sub-devices through the PCIe Switch so that SmartSSD adeptly manages SSD reads/writes, FPGA computations, and FPGA DRAM reads/writes well. Beyond device driver commands, SmartSSD exhibits support for internal data movement along the data path connecting its NVMe SSD and FPGA DRAM via the PCIe Switch—a mechanism denoted as P2P (peer-to-peer) communication.

Given the computational units' ability to directly access data in the storage medium leveraging the internal bandwidth of SmartSSD, data processing bypasses the complex operating system I/O stacks and interconnect buses. In data-intensive systems, offloading computational tasks to SmartSSD significantly reduces data movement, resulting in lower energy consumption and a corresponding reduction in host CPU and bus bandwidth utilization. Presently, there exists a substantial body of research dedicated to exploring the application scenarios for SmartSSD [18, 29].

3 Motivation and Challenge

To understand the characteristics of KV separation, we first conduct a set of experiments on the widely used KV separated systems Titan, BlobDB, and DiffKV in comparison to traditional LSM store RocksDB with default parameter settings on a server equipped with two 18-cores Intel Xeon Gold 5220 2.20GHz CPUs with two-way hyper-threading, a 64 GB DDR4-2666 memory, and a Samsung SmartSSD without offloading function. We use a total 20 GB dataset containing 20million KV pairs of the same size and conduct 200 million insert operations under 16 parallel client threads. For each KV pair, the key size is 24 bytes and the value size is 1 KB.

Table 1: The throughput, 99% latency, space, compaction I/O, write stall results of RocksDB, BlobDB, Titan, and DiffKV

	RocksDB	BlobDB	Titan	DiffKV
Throughput (ops/s)	23051	27151	18247	28484
99% Latency (ms)	1.307	0.657	1.888	0.591
Space (GB)	29	155	31	73
Compaction (GB)	1110	78	47	443
Stall (s)	1230	431	0	288

In order to comprehensively characterize KV separation

systems, we not only evaluate the throughput and 99% latency metrics of RocksDB, Titan, BlobDB, and DiffKV, that are typically used to show performance, but we also add space usage, compaction I/O, and write stall to the list of metrics to be considered in a holistic way, which are often unnoticed by previous researchers. The space metric shows the storage cost of a KV-separated LSM store, which is critical for real-world data storage scenarios, and constrains the extent to which the system's space redundancy is magnified. The compaction I/O and write stall metrics, on the other hand, show the amount of I/O and overhead of compaction, which are two parameters that can consider whether the core purpose of the KV separation technique has been effectively realized. The results are displayed in Table 1. We further visualize and quantify these metrics in Figure 3 to highlight the strengths of these systems. We quantify the best and worst result in each metric as 1 and 0 respectively (larger results are better for throughput and smaller for other metrics) and normalize the other results into this interval.

BlobDB and DiffKV demonstrate similar performance. Compared to RocksDB, BlobDB and DiffKV have 12.5% and 27.5% higher throughput, and 71.6% and 78.9% lower 99% tail latency, respectively, demonstrating the significant benefit of Compaction-triggered GC by reducing competition for LSM reads and writes. While BlobDB and DiffKV achieve higher performance, they face data volume and computational bottlenecks. In terms of space overhead, receiving 200GB updates in a 20GB data domain, RocksDB requires only 9GB space to temporarily store expired data, but BlobDB and DiffKV require 135GB and 53GB, respectively, which is a significant increase in space amplification. Regarding compaction and write stall, BlobDB and DiffKV have improvements over RocksDB, showing 60% and 93% decreases in compaction I/O, and 77% and 65% decreases in write stall, which demonstrates the effectiveness of KV separation on resolving compaction bottleneck. However, we see that Titan is much better than BlobDB and DiffKV in terms of space usage, only 2GB higher than RocksDB. Titan shows the best results in terms of compaction I/O and write stall.

The reason for the increased overhead of compaction in DiffKV and BlobDB is that the GC and compaction, which are originally executed asynchronously in the Direct GC method, are migrated from the GC process to the compaction process in the Compaction-triggered GC method. This significantly lengthens the processing latency of compaction, which leads to the write stalls. On the other hand, the reason for the significant increase in redundancy space is that the key involved in a single compaction may not cover all the data in a Blob. That is, after a Compaction-triggered GC, part of the data in a Blob is migrated to the new location, while there is still much data that cannot be migrated because it corresponds to other SSTable. This part of the data still retained in the old Blob needs to wait for the subsequent compaction of the corresponding SSTable before it can be processed, so the old Blob continues to occupy the space, resulting in the redundant data can not be deleted in a timely manner. In contrast, Direct GC does not have this deletion deferral and deletes the old Blob directly after one GC migration. But strangely, even with the advantage of the KV separation architecture and Direct GC, Titan has a 18.3% drop in throughput compared to RocksDB. In summary, we get the first observation that **the performance results of existing KV separated systems exhibit trade-offs and do not achieve simultaneous advantages in throughput, tail latency, and space usage**.

Table 2: Performance analysis of Titan

	Throughput	**99% Latency**
Titan	18247	1.888
Titan w/o GC	30591.3	0.583
Titan (16 cores)	14070.4	1.854
Titan w/o GC (16 cores)	28769.6	0.848

While these optimization efforts of DiffKV and BlobDB for KV-Separated LSM storage systems trade off the timeliness of the GC policy and the amount of GC data for excellent throughput and tail latency, they instead increase the overhead of compaction and greatly waste storage space. However, we note that Titan effectively achieves the expected performance benefits of KV separation in these two areas, but curiously, Titan's throughput performance and tail latency do not achieve the expected performance benefits of the KV separation mechanism. Based on DiffKV and BlobDB in the direction of KV separation optimization is to optimize the GC operation, we further directly turn off the GC option in Titan to evaluate the impact on the system. When turning off GC, Titan still inserts values directly into Blob files and keys and value offsets into the LSM index when receiving write operations. Compaction operations are still performed on the LSM tree, but no GC operations are performed in Blob files, thus the Value region will be infinitely inflated. Table 2 depicts the result that, without using GC, Titan achieves a breakthrough in performance, improving 67.7% in throughput and reducing 69.1% in tail latency, indicating that GC operations are still a performance bottleneck for Titan.

In order to explore the cause of the GC bottleneck, we examine the resource competition of the GC. We test Titan again in a simulated CPU resource constrained environment, where we set the number of available CPU cores to 16 (same number of parallel request threads). As shown in Table 2, there is no significant change in Titan w/o GC (16 cores), dropping only 6.0%, meaning that the overhead of compaction in KV separation is reduced to a very low level. However, Titan (16 cores) further encounters a sudden drop of 22.9% in throughput performance. The strict GC policy in Titan, while keeping Titan's data tightly characterized, exacerbates the bandwidth competition with foreground reads and writes.

To further verify whether the overhead of GC has a performance impact due to contention for bandwidth and compute resources, and to locate where the competition arises, we next

Table 3: GC time breakdown of Titan

Read value	Read LSM	Write value	Write LSM	Extra
5.31%	25.3%	2.18%	65.93%	1.28%

analyze the GC operation with an event trace. Table 3 shows the time consumption at each stage of the GC process. The results show that most of the latency of GC operations lies in reads and writes at the LSM index. Reading LSM determines the KV pair validity at GC time, while writing LSM updates the latest offset position. It should be noted that each time the LSM is written, not only to write the pair of key and offset, but it needs to read LSM first to check that a newer version is not written at GC time. As a result, due to the lack of optimization in Direct GC by scanning all candidate Blob files, the extensive index data I/O, as well as value file migration during the GC process, compete for bandwidth and CPU runtime resources with system read and write requests, leading to a significant decrease in throughput performance.

Upon analyzing Titan, we identify that the primary bottleneck in Direct GC arises from index data I/O competing with system read and write requests for compute and bandwidth resources. Although Compaction-triggered GC partially alleviates this issue by integrating the GC process into compaction, thereby reducing index data I/O and enhancing throughput and tail-latency performance, it does so at the cost of the timeliness associated with Direct GC. Consequently, a large volume of expired values continues to occupy redundant space, as the scope of compaction fails to encompass all expired data. This limitation hinders the potential benefits of GC offloading due to inherent scheduling inefficiencies. Fortunately, we observe that **Direct GC operations can be optimized through near-data processing on SmartSSD, overcoming the trade-off dilemma to simultaneously achieve optimal throughput, tail latency, and storage efficiency**. The key advantage of SmartSSD lies in its ability to bypass the PCIe bus during data movement, effectively avoiding performance degradation caused by GC bandwidth competition. Additionally, the FPGA computational units within SmartSSD provide an ideal environment for implementing the GC process. Treating value file data as sequentially independent streaming data enables highly parallelized pipeline optimizations. Each stage of the GC process—data encoding/decoding, filtering validation, merging, and CRC validation—can be independently implemented and optimized in hardware. This modular approach enhances parallelism through precise orchestration. Thus, by offloading GC tasks to SmartSSD, we fully mitigate bottlenecks in host resources, reduce request latency, and improve energy efficiency. This design harnesses the distinctive capabilities inherent in SmartSSD, enabling the direct execution of computational tasks at the storage device, significantly boosting system performance and overall efficiency.

While the application of SmartSSDs in LSM storage lacks dedicated research, we contend that SmartSSDs have the potential to effectively offload GC in KV separated systems. Nevertheless, several challenges hinder the efficient offloading of GC operations in KV separated systems on SmartSSDs: (1) **Read Back Travel**: Complexity arises in the implementation of the logic for traversing query validity during GC operations, as replicating the same logic in FPGA is challenging due to the hardware restriction against querying back host. Although PinK add software optimizations for back-checking under KV-SSD, it is still a challenge to efficiently utilize SmartSSD internal bandwidth and bypass the back-checking requirement. (2) **Resource Limitation**: The computational resources within SmartSSD, though powerful, are not comparable to those in the host. Notably, tremendous pressure imposed on FPGA DRAM during large file read/write operations in the GC process requires fine management. Furthermore, constraints on other hardware resources (Flip-Flops, Look-Up Tables, Block RAM) are contingent upon the level of synthesis optimization and the count of implementation units. Thus, it becomes imperative to institute efficient control over resource utilization. (3) **Extra Data Movement**: Inherent to the nature of the GC process, a fraction of I/O operations must be transmitted from hardware back to the host. While inevitable in the current design, this component can induce channel congestion and impact GC performance during bursts of I/O. Addressing this concern necessitates a design strategy to reschedule the movement of data between hardware and host.

4 Design and Implementation

Figure 4 depicts the architecture of AegonKV, which is built upon the latest version of Titan and represents the first implementation of a GC offloading strategy on SmartSSD. In AegonKV, we retain and extend the host-side memory structure and basic dataflow logic from original architecture. We implement GC logic units on the computational devices of SmartSSD, where we take advantage of SmartSSD native in-device data path capabilities and re-optimize the computation process to achieve more efficient GC performance.

To tackle the challenges associated with data rereading, we propose an extension of the utilization of compaction results within the GC Manager. This extension involves the transformation of the garbage ratio of each file into valid flags for individual KV positions within the file. To mitigate the computational overhead associated with these flags, we introduce a ValidMap data structure characterized by low computational cost during both updating and utilization in compaction and GC processes. The GC Manager plays a pivotal role in efficiently managing the validity of data during GC operations. **Moreover, to overcome the limitations posed by the finite resources of SmartSSD and to effectively leverage idle resources for data transfer and computation**, we architect three distinct modules within the GC Scheduler: I/O control, CU control, and Meta installation. These modules are tasked with unified scheduling and orchestration of data

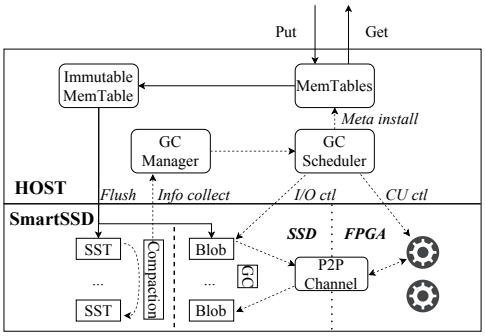

Figure 4: System Overview

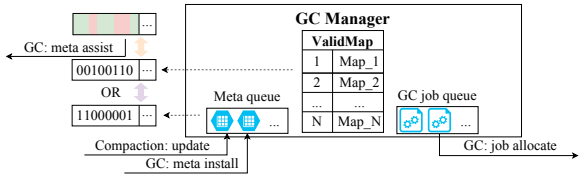

Figure 5: GC Manager

pathways, resource allocation, and data movement, thereby optimizing resource utilization. **Furthermore, to address the bandwidth contention issue arising from additional data movement**, we introduce a method for the direct bulk installation of GC metadata without validation. To ensure the integrity of the write-back data, we incorporate adaptive modifications to both read and compaction operations.

4.1 GC Manager

Figure 5 shows the GC Manager design which serves as the bridge connecting compaction and GC. Specifically, it collects essential information from compaction and prepares the necessary data structure, ValidMap, for hardware GC. Thus GC Manager implements a holistic view of Blob files within the software layer, providing room for extended scheduling.

The merging process in each compaction filters out the currently visible expired key-value pairs, which is unexplored information that can be leveraged. In AegonKV, for each Blob file, we create a Bitmap data structure [30] called ValidMap to indicate the validity of data at each offset point (bit "0" and "1" mark valid and invalid, respectively). To construct ValidMap, we add an indication of the sequential order of each storage item in the Blob file within SSTable. Since each KV data only requires a single bit in ValidMap, taking a standard 32MB Blob file as an example, when it contains 24B+1KB of KV data, the theoretical space required is 4KB. In implementation, we create a ValidMap for each Blob file, when Blob files increase or decrease, the space allocation and release of the corresponding ValidMap is handled by the system's dynamic memory allocation interface. Because of its small data volume and excellent traversal and lookup capabilities, ValidMap serves as a suitable auxiliary data structure for GC in hardware implementation.

GC Manager oversees all ValidMaps corresponding to Blob

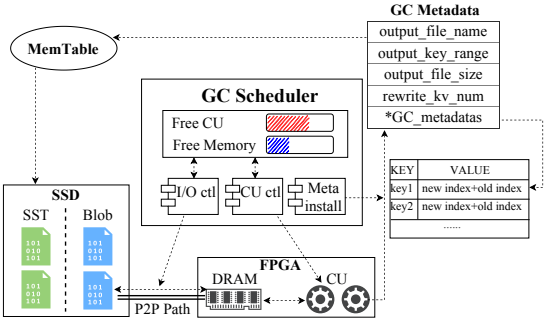

Figure 6: GC Scheduler

files. At the end of each compaction, all filtered data is passed to GC Manager during the callback. When GC Manager receives information about invalid data, it initiates the update process: traversing the data and constructing a temporary ValidMap (only the corresponding bits that are filtered are set to 1), and then performing a bit-wise OR operation with the current ValidMap, which replaces the bits "0" that flag valid with invalid "1", to complete the update.

4.2 GC Scheduler

As shown in Figure 6, we design several key units in GC Scheduler to efficiently drive the hardware for GC operations: *I/O Control Unit (I/O ctl)*, *Compute Unit Control Unit (CU ctl)*, *Metadata Install Unit (Meta install)*. These units work together to enable efficient GC operations on SmartSSD while minimizing the impact on the overall system performance.

The *I/O Control Unit* makes full use of the SmartSSD internal data path for GC. GC operations involve data transfer between SSD and DRAM, including reading and writing Blob files over SmartSSD internal P2P path, as well as the transfer of ValidMap and write-back metadata over the system bus. The hardware's asynchronous command allocation and transmission are used to enhance parallel I/O operations among different paths and tasks, all while cleverly avoiding various concurrency errors. It is important to note that SmartSSD P2P transfers use a basic unit of 4KB. To coordinate with this limitation, we have additional padding in the Blob files to align them with the 4KB boundary.

Given the limited hardware resources of SmartSSD, it is not possible to deploy an unlimited number of CUs, even the hardware memory is limited to 4GB. Effective allocation of resources is crucial while offloading. Therefore, we move CU scheduling to the software layer for simplicity and flexibility. The number of CUs in the hardware is determined during hardware compilation. As long as various hardware resource quotas are met, we aim to deploy as many CUs as possible. We use this parameter as the quota for the number of GC tasks that can be controlled. On the software side, we manage the number of CUs in use and the allocation of FPGA DRAM memory for each CU. The minimum theoretical memory requirements for each CU include input files, reserved

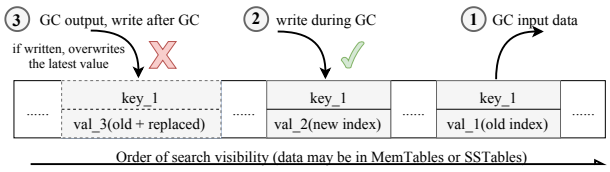

Figure 7: Version conflict of GC write-back

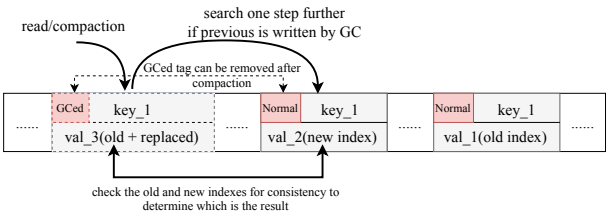

Figure 8: GC write-back state

output files, and a temporary area for intermediate calculations. We determine the size of the retained output file based on the garbage rate multiplied by the input file size, since the garbage rate corresponds to the results of the ValidMap judgments. In addition to resource allocation, the *CU ctl* Unit is responsible for starting and ending a CU. This process is asynchronous, and when a CU finishes execution, a callback function is triggered to pass the result to the *Meta install* Unit.

The results of GC need to be installed into the LSM structure, including the new addresses of the migrated KV data and the basic information of the new Blob files (key range, size, data volume, etc.). The lack of coordinated scheduling for metadata writes in previous work is a significant factor contributing to GC performance issues. The problem with metadata writes is the intense competition from concurrent GC results for the write installation interface. Considering that the installation priority of GC results is lower compared to tasks like compaction, we design a buffer queue within the *Metadata install* Unit. This buffer queue helps coordinate the concurrent competition of multiple tasks into a unified channel and maintains efficiency through batch flushing.

4.3 GC Friendly Metadata Install

Although the metadata install unit optimizes the write-back process with batch processing, it still needs to perform a lookup on each metadata to avoid the situation depicted in Figure 7, where an entry containing old data is inserted during the GC process and overwrites the latest version which is written during GC. However, this lookup process causes serious bandwidth contention overhead. Extending the idea of delayed updates in PinK, we further propose to batch insert the metadata directly into the MemTable without validation in order to achieve performance-intrusive GC write-back. However, doing this directly introduces data consistency issues, which are also not addressed in PinK [31]. Since the original purpose of validating metadata is to avoid overwriting newly inserted data during the GC process, resulting in the older data having a newer version, so that the get operation would return the old version data, and the compaction operation would filter out the newly inserted data.

To cope with this data consistency problem, we choose to defer the lookup validation process to the get and compaction processes involving the corresponding key. In the Get process, the first time the Key is looked up, it may be normal data or GC metadata (the difference between normal data and GC metadata is that the metadata additionally stores the address of the old value). In order to distinguish between normal data and GC metadata, we add a state, the GC write-back state, to the flag which is originally used to distinguish whether it is KV separated. When query to the normal data, there is no problem of data version, the latest version of the data is written directly by insert operation. But when query to the metadata as depicted in Figure 8, we are not sure whether a new key is written during GC process. For this reason, we continue to query backward to an older version of the data, and compare its value address to the original address stored in the metadata. If the same, we can determine that there is no new key written during the GC process; and if it is different, then this older version data is inserted during GC, so this data is supposed to be the latest and should be read and returned.

This situation is also encountered during the merge process in compaction, where we analogously adopt the strategy of continuing to read the next version of the data backwards. The impact of deferred validation is that redundant matching is required to determine the latest version, for this reason we promote metadata to normal data in compaction to reduce the frequency of additional read situations. When compaction encounters metadata and finds that the next older version of the key exists, if the older version of the data is actually written later (same judgment logic as in Get operation), then the metadata is filtered and the older data is written. If the metadata is already up-to-date, then we promote the metadata to normal data (removing the old value address stored in the metadata) and write it back. Due to the promotion mechanism, the metadata redundancy can be controlled by the system's garbage rate and the amount is small in the LSM. Therefore, the overhead of deferred validation is small.

4.4 Compute Unit for GC

On the FPGA side, a computing logic on board is referred to as a CU. When resources are abundant, multiple CUs implementing the same logic can coexist within a single computing core. In hardware implementation, we abstract the process into three separate stages: *Input Decode*, *Data Compute*, and *Output Encode* as shown in Figure 9. Within each stage, we implement the corresponding logic and optimizations using Vitis-HLS development flow [32].

Each CU's input includes several Blob files waiting GC, along with the ValidMap corresponding to each file. The key-value pairs within the Blob files are adjacent to each other, with offset boundaries determined by the data size, making

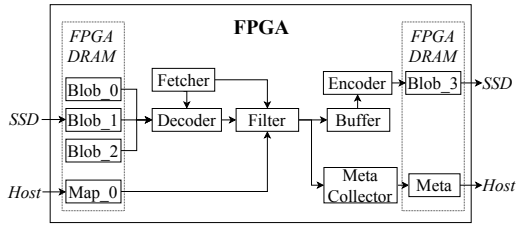

Figure 9: Compute Unit for GC

the hardware implementation highly parallel. We notice that the number and unit size of input Blob files can change as system tuning progresses, and we do not hard-code limitations on the number or total size into the hardware input ports as previous works do [33, 34]. Instead, we configure them as unlimited stream-type inputs. On the other hand, ValidMaps are encoded using the most common character data, enabling efficient retrieval of flag information at the corresponding positions on the hardware. Thus, a highly parallel-optimized CU is capable of meeting the performance requirements for input decoding in practice.

The actual computation in GC is divided into two parts in our implementation: filtering and fetching. The Filter module uses the ValidMap structure to traverse the character flags in a data-stream-like manner, producing signals for filtering or retaining data that are then passed to the Fetcher module. The Fetcher module guides how data is fetched. Because there is no need to copy and migrate expired data again, when the Filter module indicates that the current KV is invalid, it does not retrieve the KV data from the Decoder; it simply advances the current offset pointer to the start of the next KV.

The migration of KV data from the original file to the new file does not actually require modification of the entire data. The Fetcher in this process only needs to move the data between the start and end positions of the KV data to the next stage to achieve the correct logic without parsing the various data structures within it. However, to ensure correctness, we have to add a small computational overhead for breaking down the original CRC result data and performing CRC verification to ensure data integrity.

The output of the CU includes information that needs to be directly installed on the SSD and information that needs to be rewritten to the LSM MemTable. The generation of the new Blob file corresponds to the decoding process, and it directly retrieves KV from the result passed by the Fetcher module and arranges it in the file format for writing to the designated area without host-side intervention. On the other hand, to synchronize file writing management on the host side, we collect these relevant metadata during the process of encoding. This metadata includes the number of KV pairs, file size, the new and original addresses, and size of each KV pair. Then we notify the *Meta install* module of the *GC Manager* to complete the subsequent metadata write-back job, and the GC process on the hardware side is completed.

5 Evaluation

In this section, we evaluate the overall performance of AegonKV on the YCSB benchmark and production workloads, and compare it with three state-of-the-art KV separated systems, Titan, DiffKV, BlobDB, the traditional LSM storage RocksDB, and Titan without GC as ideal case target.

5.1 Experimental Setup

Testbed. We conduct experiments on the same server in the motivation experiment which is equipped with two 18-cores Intel Xeon Gold 5220 @2.20GHz CPUs with two-way hyper-threading, a 64 GB DDR4-2666 memory, and a Samsung SmartSSD containing a Xilinx's UltraScale+ FPGA and a 3.84TB SSD. The server runs Ubuntu 20.04.4 LTS (GNU/Linux 5.8.0-43-generic x86_64).

Workload. First, we use the YCSB-cpp [35] tool to generate the YCSB workflow, where the key and value sizes are 24B and 1KB, respectively. Second, we use parameters of db_bench suggested by RocksDB [10] to generate Social Graph realistic simulation workflow. The average key and value sizes are 48B and 228B respectively. Social Graph workflows contain 55% get, 44% set, and 1% scan operations. Third, we use three of Twitter open source traces [36], namely cluster 39, 19, and 51, to replay cache cluster workloads based on the experience of previous works [37], and each load replays the first 500 million of data during evaluation. Cluster 39 represents write-intensive workload, which contains 6% get and 94% set operations. The average key size is 41B and value size is 80B. Cluster 51 represents read-intensive workload, which contains 90% get and 10% set operations. The average key size is 44B and value size is 221B. Cluster 19 represents mixed workload, which contains 75% get and 25% set operations. The average key size is 42B and value size is 101B. The default number of client threads for all YCSB and realistic workloads is 16, and number of operations is 200 million when not specifically specified.

Configuration. We conduct evaluations using the latest versions of the comparison systems. For LSM's basic parameters, shared across all databases, we standardize the following settings: 64MB MemTable, 8MB SSTable, 32MB Blob, 64MB GC Batch, and KV separation for all data sizes. Regarding the work threads, we follow previous work, opening 9 compaction threads, 3 flush threads, and 8 GC threads. Compression and WAL functionalities are turned off. For Titan, another GC-related adjustable parameter is garbage ratio, and we use the default value of 0.5. For DiffKV, we utilize the recommended parameters from the original paper, enabling level merge, range merge, and setting max_sorted_runs to 10. Both AegonKV and DiffKV are built on Titan, but BlobDB employs a different GC parameter system than Titan. For fairness in testing, we adjust the preset parameters to make its space occupancy similar to Titan.

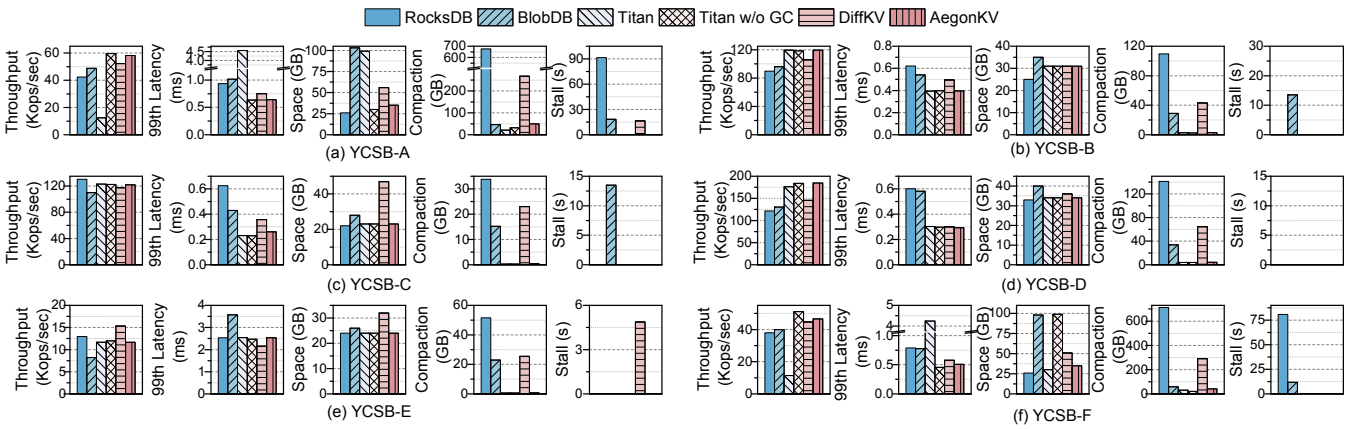

Figure 10: The throughput, 99% latency, space, compaction I/O, and write stall results of RocksDB, BlobDB, Titan, Titan w/o GC, DiffKV, AegonKV under the YCSB benchmark. YCSB A to F represent: A (50% reads, 50% updates), B (95% reads, 5% updates), C (100% reads), D (5% inserts, 95% reads of recent records), E (95% scans, 5% inserts), and F (50% reads, 50% read-modify-writes).

5.2 YCSB Workloads

Figure 10 shows the throughput, 99% tail latency, space, compaction I/O, and write stall for RocksDB, BlobDB, Titan, Titan w/o GC, DiffKV, AegonKV under the YCSB. We configure this workload with 20 million randomly loaded KV pairs and 200 million transactions under zipfian distribution.

Throughput. In comparison to BlobDB, Titan, DiffKV, and RocksDB, AegonKV has exhibited throughput enhancements ranging from 1.11 to 4.66 times in write-intensive workloads (A and F) and closely approaches the scenario of no GC overhead (Titan w/o GC). We find that RocksDB's performance lags behind the most of KV separated systems, demonstrating that KV separation architectures have a clear advantage in handling mixed workflows of reads and writes with large key-value pairs. Titan's Direct GC approach shows significant problems because the bandwidth contention caused by intensive GC hinders the system's ability to handle frequent requests. AegonKV effectively mitigates this problem through GC offloading, which entails the significant offloading of I/O from the value file to the near-data side. Furthermore, AegonKV optimizes the GC process by expediting GC index query requests to construct the ValidMap data structure and batch inserting GC metadata without validation. Meanwhile, AegonKV slightly outperforms BlobDB and DiffKV due to the significant overhead associated with the Compaction-triggered-GC of BlobDB and DiffKV. It is worth noting that AegonKV's throughput closely aligns with, albeit slightly lower than, Titan w/o GC. Despite the effective offloading, a minor portion of GC metadata in AegonKV must traverse the foreground interface, contributing to throughput competition.

The performance of AegonKV closely aligns with other systems in read-intensive workloads (B, C, and D). However, there is additional overhead due to the supplementary operations: AegonKV requires maintenance of the ValidMap, while DiffKV and BlobDB necessitate periodic organization of the Value region. Consequently, in a read-only environment, the performance of AegonKV, DiffKV, and BlobDB is not as optimal as that of Titan and Titan w/o GC, with a performance loss of 1.0%, 4.3%, and 10.5% respectively. Exceptionally, in YCSB-E, DiffKV undergoes meticulous optimization by introducing additional GC frequency to regulate the Value region systematically, thereby achieving best scan performance.

Tail Latency. Under write-intensive workloads (A and F), AegonKV achieves the lowest tail latency, which is reduced by 14.7%, 85.9%, 31.2%, and 36.8% compared to DiffKV, Titan, RocksDB, and BlobDB, respectively. The performance and causes of tail latency are basically the same as throughput, with write requests competing for bandwidth due to Titan's GC causing higher tail latency, and RocksDB, BlobDB, and DiffKV's compaction causing writes to stall and thus causing tail latency. AegonKV, on the other hand, removes GC-related overheads from the critical path via efficient offloading, similar to the results of the ideal state Titan w/o GC. For read-intensive workloads (B, C, and D), the difference in tail latency is small across all systems, due to the fact that the KV separated read streams are essentially the same across systems and GCs are not triggered frequently; thus, tail latency performance is best for the index-optimized DiffKV under the scan workflow.

Space Usage. The storage space results show the space usage at the end of the run, so the baseline space is assumed to be the 20GB of data initially loaded. Under write-intensive workloads (A and F), Titan and AegonKV require only 0.5-0.75 times more additional redundant space throughout the update process, due to the real-time monitoring of file garbage accumulation levels under the Direct GC policy and the timely GC response, which in turn brings their space usage close to that of RocksDB, which handles data compaction in real-time. On the other hand, the Compaction-triggered GC policies of BlobDB and DiffKV require much larger space, with 1.8x and 4.15x redundancy, respectively. Under read-intensive work-

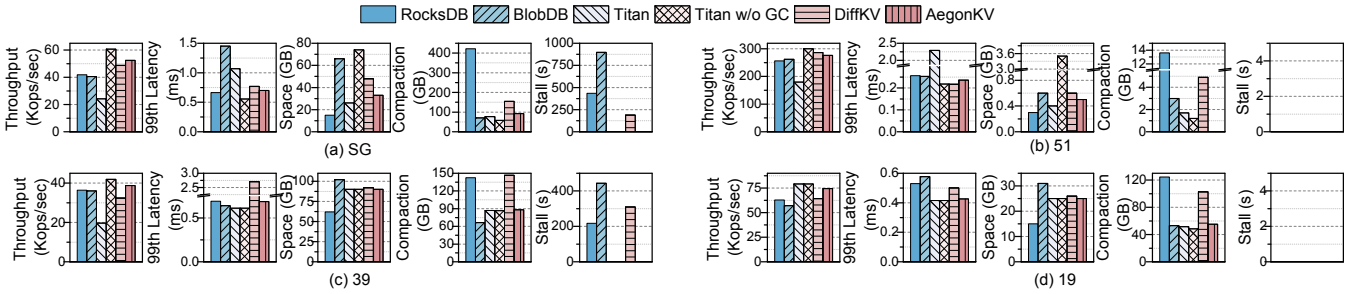

Figure 11: The throughput, 99% latency, space, compaction I/O, and write stall results of RocksDB, BlobDB, Titan, Titan w/o GC, DiffKV, AegonKV under production workloads. SG stands for Social Graph, and 39, 19, and 51 are cluster numbers respectively.

loads (B, C, D, and E), the redundancy space usage is very similar for each KV separated system due to the absence or low percentage of writes. However, we find that DiffKV and BlobDB space is larger than Titan w/o GC (under YCSB-A and F, BlobDB is also larger than Titan w/o GC). This is because Compaction-triggered GC generates a large number of intermediate files, and once compaction is performed, value data migration occurs, and the original Blob files cannot be deleted immediately (and in the case of mixed read operations, some may not have been migrated yet).

Compaction I/O. Compared with the traditional LSM RocksDB, the amount of compaction data for KV separation is significantly reduced because there is no need to read the actual Value data into the compaction process. Compared to Titan w/o GC, AegonKV and Titan add 136% and 51% extra overhead, respectively. This is due to the fact that Titan's GC writes the new address index back to the LSM, adding some compaction processing, whereas AegonKV uses the GC friendly metadata install optimization, whose write back to the LSM does not sift out the failing data, so it writes back to the LSM a bit more. Due to Value data relocation by Compaction-triggered GC, DiffKV and BlobDB show a significant increase in compaction I/O, which is 5.4-14.4 times higher than AegonKV. While BlobDB exhibits similar results to DiffKV under read-intensive workloads, it curiously does not show a significant increase under write-intensive workloads. We find that the GC of BlobDB is only performed on the oldest 50% data, which means that a large amount of newly written data is not immediately included in the GC migration, leading to further amplification of redundant data space on the one hand, and the increase in the amount of compaction data on the other hand is not significant.

Write Stall. Compared to RocksDB's large amount of write stall time under write-intensive workloads, AegonKV and Titan have no write stalls, thus KV separation solves the write stall problem of compaction well. However, in order to optimize the throughput performance of Direct GC, DiffKV and BlobDB's Compaction-triggered GC strategy in turn increases the frequency and overhead of compaction, about 18%-20% of the write stall time of RocksDB.

Summary. We give the capability performance of each KV separated system under the YCSB benchmark in Figure 12(a),

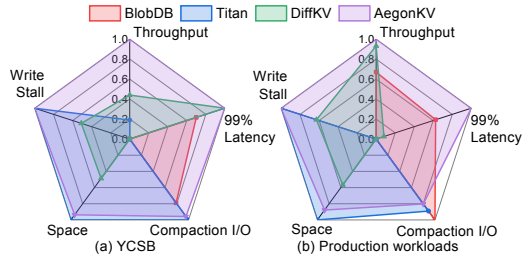

Figure 12: The throughput, 99% latency, space, compaction data, write stall capability comparison of BlobDB, Titan, DiffKV, AegonKV under YCSB and production workloads

which is constructed in the same way as in Figure 3. AegonKV implements a full range of optimizations for the KV separation architecture in terms of throughput, tail latency, space usage, compaction I/O, and write stall.

5.3 Production Workloads

Figure 11 shows throughput, tail latency, space usage, compaction I/O, and write stall results under the production workloads. For the Social Graph workflow, the trend in performance results across systems is generally consistent with YCSB-A. AegonKV increases throughput by a factor of 1.07 to 2.16 and reduces tail latency by 10%-52% compared to other KV separated systems. AegonKV inherits the advantages of Direct GC in terms of space usage, compaction I/O, and write stall metrics, reducing redundant space usage by 31%-55% and compaction I/O by 40%-78%, and maintaining 0 write stall. AegonKV shows the best level of performance in all indicators, making AegonKV well suited for real-world production scenarios with a mix of reads and writes.

For the Twitter cache cluster workflow, under the write-intensive workload (cluster39), AegonKV achieves 7.6%-97.7% improvement in throughput, 38%-40% reduction in compaction I/O and have 0 write stall, implying the effectiveness of AegonKV's Direct GC policy and GC offloading strategy in write-intensive scenarios. However, we observe that all the systems perform very closely in terms of tail latency and space occupation. After re-analyzing the workflow we find that the KV pair insertion repetition rate for cluster 39 is small, meaning that the GC trigger rate is very low and the GC optimization gain of AegonKV is scaled down. Under

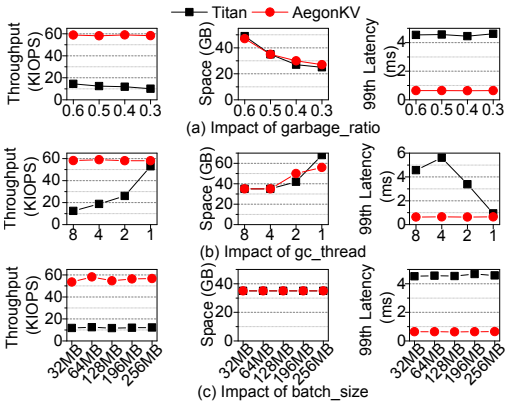

Figure 13: The throughput, space, and 99% latency of Titan, and AegonKV varies with garbage_ratio, gc_thread, and batch_size

the mixed workload (cluster19) and the read-intensive workload (cluster51), we observe similar results for the YCSB read workloads where all systems are close in all aspects of performance. Unlike the results of YCSB read workloads, in addition to the low GC frequency due to intensive read operations, the value size in these two workloads also affects the read performance. In cluster51 and cluster19, which contain a large amount of small data (less than 100B), since the read overhead of small data is insignificant, this further reduces the proportion of the difference in read operation overhead between different systems, resulting in a small difference in read performance. We additionally observe that the compaction I/O of RocksDB and DiffKV is 1.7 times higher than the other systems. The reason for the large compaction I/O of DiffKV and RocksDB is still that the GC of DiffKV needs to initiate compaction frequently, and the traditional LSM RocksDB itself has high compaction overhead.

The capability comparison under the production workloads is shown in Figure 12(b). In a similar trend to the YCSB results, AegonKV remains excellent in all five metrics.

5.4 Overhead Analysis

Resource Consumption. Table 4 report the hardware resources required for each CU, including CLB (*Configurable Logic Blocks*), LUT (*Lookup Table*), Flip Flop, and BRAM. Without considering memory overhead, CLB resource is the main factor limiting the number of CU deployments. We fully utilize the hardware resources of SmartSSD and deploy 8 CUs, which is enough to support the system to run normally.

Table 4: Hardware resource utilization

	CLB	LUT	Flip Flop	BRAM(MB)
1 CU	-	19704	31404	10
Used total	63817	292661	466312	445.5
Available	65340	522720	1045440	984
Utilization	97.6%	55.9%	44.6%	45.2%

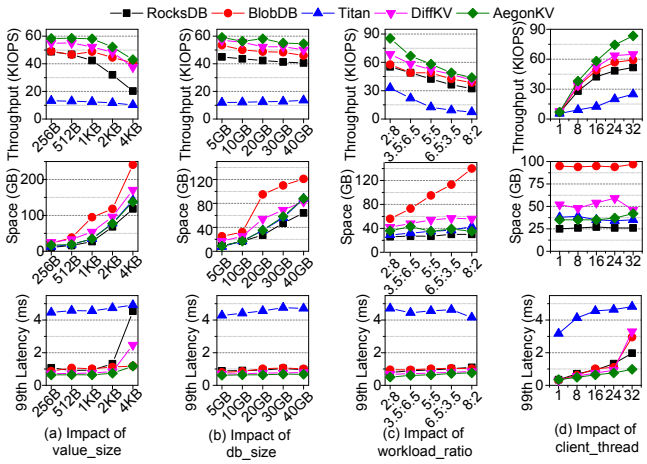

Figure 14: The throughput, space, and 99% latency of RocksDB, BlobDB, Titan, DiffKV, and AegonKV varies with value_size, db_size, workload_ratio (write:read), and client_thread

CPU Usage. We calculate the normalized CPU utilization by dividing the total CPU usage of the monitored processes by the current total number of active threads, including the thread count under offloading conditions. The results are 95.52, 97.18, 90.4, 96.97, 97.16, 77.17 (%) for RocksDB, BlobDB, Titan, Titan w/o GC, DiffKV, and AegonKV, respectively. While other systems maintain close to 100% CPU utilization, AegonKV achieves approximately a 20% savings. The reason for this lies in the GC computation overhead in other systems, either directly calculated or migrated to the compaction phase. The GC offloading in AegonKV effectively resolves the computational bottleneck.

Power Usage. We use the PowerTop [38] and XRT tool [39] to measure CPU and SmartSSD power respectively. The energy consumption results are obtained by dividing the system throughput by the dynamic power of the CPU and SSD. The results are 571.64, 775.11, 246.82, 720.28, 901.78 (Operations/J) for RocksDB, BlobDB, Titan, DiffKV, and AegonKV, respectively. Since CPU overhead is much larger in energy consumption statistics and SSDs consume only a small fraction of the energy, by offloading CPU computations to the low-power SmartSSD computational capabilities, AegonKV achieves the highest energy efficiency, improving by 15.9% to 69.8%. The high GC overhead of Titan and compaction overhead of BlobDB, DiffKV, and RocksDB results in a dramatic increase in CPU energy consumption.

5.5 Sensitivity Analysis

Since we mainly focus on the performance of system throughput, tail latency, and space usage, we next analyze the parameter sensitivity of AegonKV and other systems under YCSB-A.

System Parameter. AegonKV is consistent with Titan in terms of GC parameter, but has a different parameter system than others, so we only compare AegonKV to Titan in terms of

system parameters. Figure 13(a) shows how the garbage_ratio parameter affects the performance of AegonKV and Titan. This parameter is used to determine the minimum garbage threshold for a file to be added to the GC queue. Even with a lower garbage ratio (meaning the less aggressive GC), Titan's performance is still poor. However, for AegonKV, as we detach GC from the critical path of the system, the throughput and tail latency performance of AegonKV remain unchanged under different garbage ratio. Regarding space occupancy, Titan and AegonKV are very close because they use the same GC triggering method. Figure 13(b) shows how the gc_thread parameter affects the performance of AegonKV. Through task offloading, an increased number of GC threads does not impact AegonKV's throughput and tail latency. The results also demonstrate that configuring more offload CUs for AegonKV does not yield additional benefits in terms of space performance. It suffices to meet a certain minimum number, aligning with 4 threads in the experiment setting. In Figure 13(c), we observe no impact on system performance and overhead as we adjust the batch_size parameter from 32MB to 256MB, as the batch size of every GC task has no direct effect on the frequency or rate of GC. *In summary, the performance advantage of AegonKV is not affected by system parameters, and it does not require excessive computational resources to maintain optimal performance.*

Workload Parameter. Next, we test the sensitivity of workload-related parameters. For value_size in Figure 14(a), smaller values imply a reduction in read and write overhead, which improves system throughput, and reduces tail latency and storage space, and vice versa. Regardless of the change in value size, AegonKV shows the best performance in terms of throughput, tail latency, and storage space, and is less sensitive to changes in value. For db_size in Figure 14(b), the dataset size has little effect on the read and write operation links of the system, because the efficiency of the read operations, which are first queried and located in the index, and the write operations, which are done in the form of append, are not affected by the actual db size. Therefore, when the dataset increases, the throughput and tail latency of each system do not change significantly. Although system space appears to increase as the dataset increases, the percentage of redundant space does not actually change significantly. For workload_ratio in Figure 14(c), we simulate various scenarios by adjusting the proportion of read and update operations. Regardless of read/write intensive or balanced scenarios, AegonKV shows the most excellent throughput and tail latency, which is consistent with the results under YCSB and real workloads. In terms of storage space, while AegonKV's space amplification is superior to other KV separation systems, the timely compaction of traditional LSM store minimizes the redundancy space requirements of RocksDB. For client_thread in Figure 14(d), by rationally offloading bandwidth-competitive GC operations and benefiting from the low compaction overhead of KV separation, AegonKV suffers the least from multi-thread conflicts and shows the best throughput and tail latency results, which indicates that AegonKV has the best multi-thread scalability. The number of operation threads has no effect on the final data space occupation, and AegonKV still outperforms other KV separated systems and slightly under-performs RocksDB, which is consistent with the results in workload_ratio. *In summary, AegonKV's performance advantages over other KV separated systems in terms of throughput, tail latency, and space overhead do not change drastically due to changes in workload parameters, and AegonKV has optimal workload adaptability and multi-thread scalability.*

6 Related Works

Recent endeavors, exemplified by DiffKV [8], SpacKV [40], Parallax [41], and SineKV [14], primarily optimize KV separation by leveraging LSM structures for value management. While this methodology facilitates the coordination of Key and Value regions through compaction, thereby distributing the centralized overhead of GC across diverse runtime periods, it inadvertently neglects the potential reintroduction of write stalls and exacerbation of read-write amplification issues stemming from heightened compaction tasks.

PinK [9, 31] introduces the KV separation architecture into the design of KV-SSD and achieves excellent performance gains. PinK mainly focuses on read-write optimized KV-SSD based LSM indexing and adds mechanisms such as hardware acceleration for compaction and software optimizations for GC. However, AegonKV focuses on GC offloading because the GC overhead in KV-separated LSM systems is significantly higher than that of compaction.

Other recent works introduce machine learning into the KV separation architecture, such as Bourbon's [42] use of learned index to accelerate lookup operations, and DumpKV's [43] use of learning methods to sense value file life-cycles for dynamic and efficient GC. In contrast, our work diverges by incorporating advanced value organization structures while adhering to Direct GC methods. Our overall goal is to comprehensively address both computational and read-write bottlenecks associated with GC through offloading.

7 Conclusion

In order to fully optimize GC performance across all dimensions of throughput, tail latency, storage overhead at the same time, we propose AegonKV. AegonKV adeptly offloads GC operations to in-storage computation SmartSSD, thereby effectively mitigating both CPU and I/O pressure. The empirical results, by comparing AegonKV with state-of-the-art CPU-based KV separated systems, demonstrate AegonKV's ability to deliver excellent throughput and tail-latency performance while keeping data movement and storage overheads minimal.

References

[1] Wei Cao, Yang Liu, Zhushi Cheng, Ning Zheng, Wei Li, Wenjie Wu, Linqiang Ouyang, Peng Wang, Yijing Wang, Ray Kuan, Zhenjun Liu, Feng Zhu, and Tong Zhang. POLARDB Meets Computational Storage: Efficiently Support Analytical Workloads in Cloud-Native Relational Database. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 29–41, 2020.

[2] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, Cong Liu, Jian Zhang, Jianjun Li, Xuelian Wu, Lingyu Song, Ruoxi Sun, Shuaipeng Yu, Lei Zhao, Nicholas Cameron, Liquan Pei, and Xin Tang. TiDB: A Raft-based HTAP Database. *Proceedings of the VLDB Endowment*, 13(12):3072–3084, 2020.

[3] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. WiscKey: Separating Keys from Values in SSD-conscious Storage. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies (FAST '16)*, pages 133–148, 2016.

[4] Helen H. W. Chan, Yongkun Li, Patrick P. C. Lee, and Yinlong Xu. HashKV: Enabling Efficient Updates in KV Storage via Hashing. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 1007–1019, 2018.

[5] PingCap. Titan. https://github.com/tikv/titan.

[6] Jianchuan Li, Peiquan Jin, Yuanjin Lin, Ming Zhao, Yi Wang, and Kuankuan Guo. Elastic and Stable Compaction for LSM-tree: A FaaS-Based Approach on TerarkDB. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21)*, pages 3906–3915, 2021.

[7] Meta. RocksDB. https://github.com/facebook/rocksdb.

[8] Yongkun Li, Zhen Liu, Patrick P. C. Lee, Jiayu Wu, Yinlong Xu, Yi Wu, Liu Tang, Qi Liu, and Qiu Cui. Differentiated Key-Value Storage Management for Balanced I/O Performance. In *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 673–687, 2021.

[9] Junsu Im, Jinwook Bae, Chanwoo Chung, Arvind, and Sungjin Lee. PinK: High-speed in-storage key-value store with bounded tails. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 173–187, 2020.

[10] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H. C. Du. Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 209–223, 2020.

[11] Juncheng Yang, Yao Yue, and K. V. Rashmi. A Large-scale Analysis of Hundreds of In-memory Key-value Cache Clusters at Twitter. *ACM Transactions on Storage*, 17(3):17:1–17:35, 2021.

[12] Chenlei Tang, Jiguang Wan, and Changsheng Xie. FenceKV: Enabling Efficient Range Query for Key-Value Separation. *IEEE Transactions on Parallel and Distributed Systems*, 33(12):3375–3386, 2022.

[13] Chenlei Tang, Jiguang Wan, Zhihu Tan, and Guokuan Li. RepKV: A Replicated Key-Value Store to Boost Multiple Indices for Key-Value Separation. In *Proceedings of the 2022 IEEE 40th International Conference on Computer Design (ICCD '22)*, pages 187–194, 2022.

[14] Fei Li, Youyou Lu, Zhe Yang, and Jiwu Shu. SineKV: Decoupled Secondary Indexing for LSM-based Key-Value Stores. In *Proceedings of the 40th IEEE International Conference on Distributed Computing Systems (ICDCS '20)*, pages 1112–1122, 2020.

[15] Ryan Nathanael Soenjoto Widodo, Hiroki Ohtsuji, Erika Hayashi, Eiji Yoshida, Hirotake Abe, and Kazuhiko Kato. NVKVS: Non-Volatile Memory Optimized Key-Value Separated LSM-Tree. *Journal of Information Processing Systems*, 30:332–342, 2022.

[16] Mingxuan Liu and Jianhua Gu. uCleaner: An Efficient Adaptive Garbage Collection Mechanism for KV-Separated LSM-Stores. In *Proceedings of the 2022 5th International Conference on Data Science and Information Technology (DSIT '22)*, pages 1–6, 2022.

[17] Mohammadreza Soltaniyeh, Veronica Lagrange Moutinho Dos Reis, Matt Bryson, Xuebin Yao, Richard P. Martin, and Santosh Nagarakatte. Near-Storage Processing for Solid State Drive Based Recommendation Inference with SmartSSDs. In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering (ICPE '22)*, page 177–186, 2022.

[18] Ji-Hoon Kim, Yeo-Reum Park, Jaeyoung Do, Soo-Young Ji, and Joo-Young Kim. Accelerating Large-Scale Graph-Based Nearest Neighbor Search on a Computational Storage Platform. *IEEE Transactions on Computers*, 72(1):278–290, 2023.

[19] Erfan Bank Tavakoli, Amir Beygi, and Xuebin Yao. RP-kNN: An OpenCL-Based FPGA Implementation of the

Dimensionality-Reduced kNN Algorithm Using Random Projection. *IEEE Transactions on Very Large Scale Integration Systems*, 30(4):549–552, 2022.

[20] Seongyoung Kang and Sang-Woo Jun. Near-Storage Accelerator for Bulk Graph Ingestion. In *Proceedings of the 2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW '23)*, pages 101–104, 2023.

[21] Chen Zou, Hui Zhang, Andrew A. Chien, and Yang-Seok Ki. PSACS: Highly-Parallel Shuffle Accelerator on Computational Storage. In *Proceedings of the 39th IEEE International Conference on Computer Design (ICCD '21)*, pages 480–487, 2021.

[22] Ahsan Javed Awan, Moriyoshi Ohara, Eduard Ayguade, Kazuaki Ishizaki, Mats Brorsson, and Vladimir Vlassov. Identifying the Potential of near Data Processing for Apache Spark. In *Proceedings of the International Symposium on Memory Systems (MEMSYS '17)*, page 60–67, 2017.

[23] Hwajung Kim, Heon Y. Yeom, and Hanul Sung. Understanding the Performance Characteristics of Computational Storage Drives: A Case Study with SmartSSD. *Electronics*, 10(21), 2021.

[24] AegonKV. https://github.com/CGCL-codes/AegonKV.

[25] Meta. BlobDB. https://github.com/facebook/rocksdb/wiki/BlobDB.

[26] Geogle. LevelDB. https://github.com/google/leveldb.

[27] Boyu Tian, Qihang Chen, and Mingyu Gao. ABNDP: Co-Optimizing Data Access and Load Balance in Near-Data Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, page 3–17, 2023.

[28] Joo Hwan Lee, Hui Zhang, Veronica Lagrange Moutinho dos Reis, Praveen Krishnamoorthy, Xiaodong Zhao, and Yang-Seok Ki. SmartSSD: FPGA Accelerated Near-Storage Data Analytics on SSD. *IEEE Computer Architecture Letters*, 19(2):114–117, 2020.

[29] Sahand Salamat, Armin Haj Aboutalebi, Behnam Khaleghi, Joo Hwan Lee, Yang-Seok Ki, and Tajana Rosing. NASCENT: Near-Storage Acceleration of Database Sort on SmartSSD. In *Proceedings of the 2021 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '21)*, pages 262–272, 2021.

[30] Chee-Yong Chan and Yannis E. Ioannidis. Bitmap Index Design and Evaluation. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data (SIGMOD '98)*, page 355–366, 1998.

[31] Junsu Im, Jinwook Bae, Chanwoo Chung, Arvind, and Sungjin Lee. Design of lsm-tree-based key-value ssds with bounded tails. *ACM Transactions on Storage*, 17(2), 2021.

[32] Xilinx. Vitis-HLS. https://docs.xilinx.com/r/en-US/ug1399-vitis-hls.

[33] Teng Zhang, Jianying Wang, Xuntao Cheng, Hao Xu, Nanlong Yu, Gui Huang, Tieying Zhang, Dengcheng He, Feifei Li, Wei Cao, Zhongdong Huang, and Jianling Sun. FPGA-Accelerated Compactions for LSM-based Key-Value Store. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 225–237, 2020.

[34] Xuan Sun, Jinghuan Yu, Zimeng Zhou, and Chun Jason Xue. FPGA-based Compaction Engine for Accelerating LSM-tree Key-Value Stores. In *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE '20)*, pages 1261–1272, 2020.

[35] ls4154. YCSB-cpp. https://github.com/ls4154/YCSB-cpp.

[36] Juncheng Yang, Yao Yue, and K. V. Rashmi. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, pages 191–208, 2020.

[37] Ashwini Raina, Jianan Lu, Asaf Cidon, and Michael J. Freedman. Efficient Compactions between Storage Tiers with PrismDB. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, page 179–193, 2023.

[38] fenrus75. PowerTop. https://github.com/fenrus75/powertop.

[39] Xilinx. XRT. https://github.com/Xilinx/XRT.

[40] Xuran Ge, Ming-che Lai, Yang Liu, Lizhou Wu, Zhutao Zhuang, Yang Ou, Zhiguang Chen, and Nong Xiao. SpacKV: A Pmem-Aware Key-Value Separation Store Based on LSM-Tree. In *Proceedings of the 19th IFIP International Conference on Network and Parallel Computing (NPC '22)*, pages 327–339, 2022.

[41] Giorgos Xanthakis, Giorgos Saloustros, Nikos Batsaras, Anastasios Papagiannis, and Angelos Bilas. Parallax: Hybrid Key-Value Placement in LSM-based Key-Value

Stores. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, pages 305–318, 2021.

[42] Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnatthan Alagappan, Brian Kroth, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. From WiscKey to bourbon: A learned index for Log-Structured merge trees. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 155–171, 2020.

[43] BilyZ98. DumpKV. https://github.com/BilyZ98/DumpKV.

A Artifact Appendix

Abstract

AegonKV is a "three-birds-one-stone" solution that comprehensively enhances the throughput, tail latency, and space usage of KV separated systems. The artifacts provide implementation source code of AegonKV, several workloads and evaluation scripts which is used to generate all the results presented in the paper.

Scope

The artifacts include our design and implementation of AegonKV presented in Section 4, as depicted in Figure 4-9. Furthermore, the artifacts comprise the scripts that enable the replication of results we present in the paper, including Figure 3 and Table 1-3 in Section 3, and Figure 10-14 and Table 4 in Section 6.

Contents

The corresponding folders for each artifact are listed below:

A_1 AegonKV

A_2 Evaluation

The role of Artifact A_1 is to present the implementation of AegonKV, and Artifact A_2 is to generate workloads and initiate performance evaluations.

In artifact A_1, the main directory contains the `README.md` file, which instructs how to build the hardware compute unit on SmartSSD and compile AegonKV on the host side. The source code related to the design of hardware side is presented in the directory `hardware`, and the code related to the host side is in the directory `src` and `lib/rocksdb-6.29.tikv/db`.

In artifact A_2, the main directory also contains the `README.md` file, which instructs how to build the YCSB, Social Graph, and Twitter Cluster workloads used in the paper and evaluate on Titan, DiffKV, BlobDB, RocksDB, and AegonKV. The workload configurations are in the directory `workload`, and the configurations for each system are in the directory `titandb` (contains Titan, DiffKV, AegonKV) and `rocksdb` (contains BlobDB and RocksDB). The startup scripts for each evaluation are in the `titandb/script.sh` and `titandb/real-workload.sh` file.

Hosting

All artifacts are available at https://github.com/CGCL-codes/AegonKV. The `main` branch has the latest contents.

Requirements

Hardware. The artifacts use a server with hardware dependencies shown below.

1. CPU: 2 x 18-core 2.20GHz Intel Xeon Gold 5220 with two-way hyper-threading
2. DRAM: 1 x 64 GB 2666 MHz DDR4 DRAM
3. SSD: 1 x Samsung SmartSSD

Software. Before running the artifacts, it's essential that you have already installed the software dependencies shown below.

1. GCC (9.4.0, https://gcc.gnu.org)
2. CMake (3.25.6, https://cmake.org)
3. zlib (1.3.1, https://zlib.net)
4. gflags (2.2.2, https://github.com/gflags/gflags)
5. Vitis (2021.1, https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vitis/archive-vitis.html)
6. Xilinx XRT (2021.1, https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/alveo/smartssd.html)
7. SmartSSD Development Target Platform (2021.1, https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/alveo/smartssd.html)
8. Python (3.8.10, https://www.python.org)